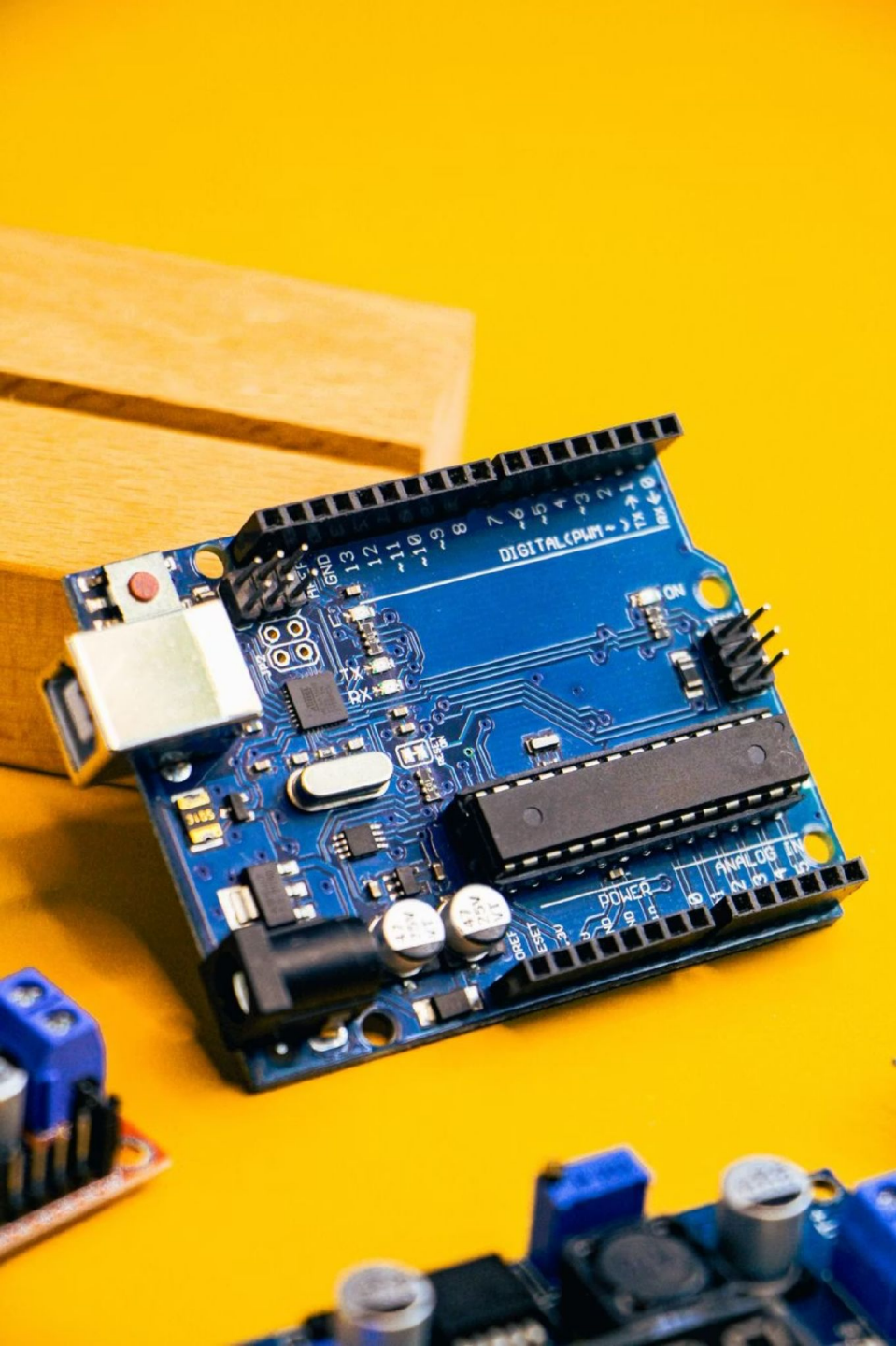




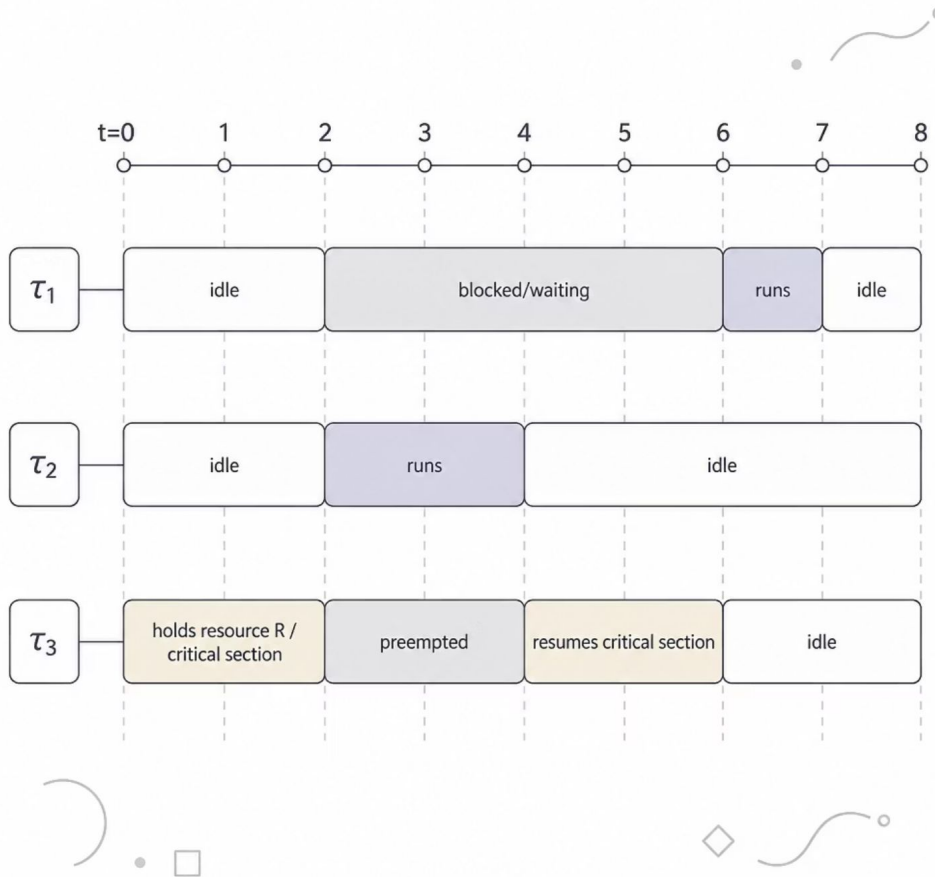
# Priority Inheritance Protocol

## Resource Access Protocols in Real-Time Systems

Nnaemeka Nnachi-Egwu · Electronic Engineering

Real-Time Systems Seminar · **Prof. Dr. Stefan Henkler**

# The Priority Inversion Problem



**Scenario:**  $\tau_3$  holds R.  $\tau_1$  arrives and blocks.  $\tau_2$  (medium priority, no R) preempts  $\tau_3$  —  $\tau_1$  stuck while a **lower-priority** task runs.

Duration: **unbounded** if many medium-priority tasks exist.



**Kopetz:** A result after its deadline is **logically incorrect**, not just late.

# The Fix: Inherit the Blocker's Priority

## Before PIP

$\tau_3$  holds R at P3;  $\tau_1$  blocked;  $\tau_2$  preempts  $\tau_3$ ;  $\tau_1$  stuck unbounded

## After PIP

$\tau_3$  inherits P1;  $\tau_2$  cannot preempt;  $\tau_3$  finishes CS;  $\tau_1$  unblocked

When  $\tau_1$  blocks on  $S_R$  held by  $\tau_3$ ,  $\tau_3$  **inherits**  $\tau_1$ 's priority:

$$p_j(S_R) = \max\{P_j, \max_h P_h \mid \tau_h \text{ blocked on } S_R\}$$

Effect:  $\tau_2$  cannot preempt  $\tau_3$  anymore. After signal,  $\tau_3$ 's priority is **restored** to nominal. Blocking bounded to  $\tau_3$ 's remaining CS length.

# pi\_wait and pi\_signal (Buttazzo §7.6.4)

Transitive inheritance propagates the highest blocked priority through the chain.

## pi\_wait(S)

- If S free  $\rightarrow$  lock it
- Else  $\rightarrow$  block caller, add to queue
- Inherit priority via Eq. (1); apply transitively

## pi\_signal(S)

- Unlock S
- Wake highest-priority waiter
- Restore holder's priority to max of nominal and remaining waiters

  $\tau_3$  holds SR and blocks  $\tau_2$ , which blocks  $\tau_1$ ; P1 is inherited up.

# Theorem 7.2: Bounded Blocking

*Sha, Rajkumar & Lehoczky, IEEE Trans. Computers, 1990*

## Blocking Bound

$$a_i = \min(l_i, s_i)$$

$l_i$  = lower-priority tasks that can block  $\tau_i$

$s_i$  = semaphores that can block  $\tau_i$

$$B_i = \min(B_i^l, B_i^s)$$

Complexity:  $\mathcal{O}(mn^2)$ . Bound is **not tight** (Buttazzo p. 222).

## Schedulability with Blocking

Liu-Layland test (Eq. 7.19):

$$\sum h$$

When inconclusive, exact Response-Time Analysis (Eq. 7.22):

$$R_i = C_i + B_i + \sum h$$

# Three-Task Example

| Task           | C | T  | D  | Prio | CS | B |
|----------------|---|----|----|------|----|---|
| T <sub>1</sub> | 1 | 6  | 6  | 1    | 1  | 4 |
| T <sub>2</sub> | 2 | 8  | 8  | 2    | —  | 4 |
| T <sub>3</sub> | 4 | 24 | 24 | 3    | 4  | 0 |

Shared semaphore  $S_R$ , ceiling =  $P_1$ .  $B_1 = 4$  (direct blocking),  $B_2 = 4$  (push-through, Lemma 7.1),  $B_3 = 0$ .

**RTA results:**  $R_1 = 5 \leq 6 \checkmark \cdot R_2 = 8 \leq 8 \checkmark \cdot R_3 = 8 \leq 24 \checkmark$

Liu-Layland inconclusive for  $\tau_2$  ( $0.917 > 0.828$ ) — RTA gives exact result.

# Five Resource Access Protocols

Buttazzo Table 7.5 — PIP column highlighted

| Property         | NPP  | HLP  | PIP        | PCP    | SRP  |
|------------------|------|------|------------|--------|------|
| Deadlock-free    | ✓    | ✓    | ✗          | ✓      | ✓    |
| Chained blocking | —    | —    | Yes        | —      | —    |
| Transparent      | ✓    | —    | ✓          | —      | —    |
| Max blockings    | 1    | 1    | $\alpha_i$ | 1      | 1    |
| Implementation   | Easy | Easy | Hard       | Medium | Easy |
| Dynamic priority | —    | —    | —          | —      | ✓    |

# Formal Verification with UPPAAL



ResourceManager automaton encoding PIP inheritance logic with atomic priority updates.

## Model Components

- **Task** template: Idle / Ready / Running / CS / Blocked
- **ResourceManager**: Free / Held / Granting
- Shared: act\_prio[3], holder, block\_dur[3]

## Verified Properties

- P1:  $A[]$  not deadlock
- P2:  $A[](\text{blocking\_tau1} \Rightarrow \text{not tau2.Running})$
- P3:  $A[](\text{block\_dur} \leq 4)$
- P4:  $E<>(\text{act\_prio}[2]==1) \text{ — inheritance reachable}$
- P5:  $A[](\text{rm.Free} \Rightarrow \text{act\_prio}[2]==3) \text{ — priority restored}$

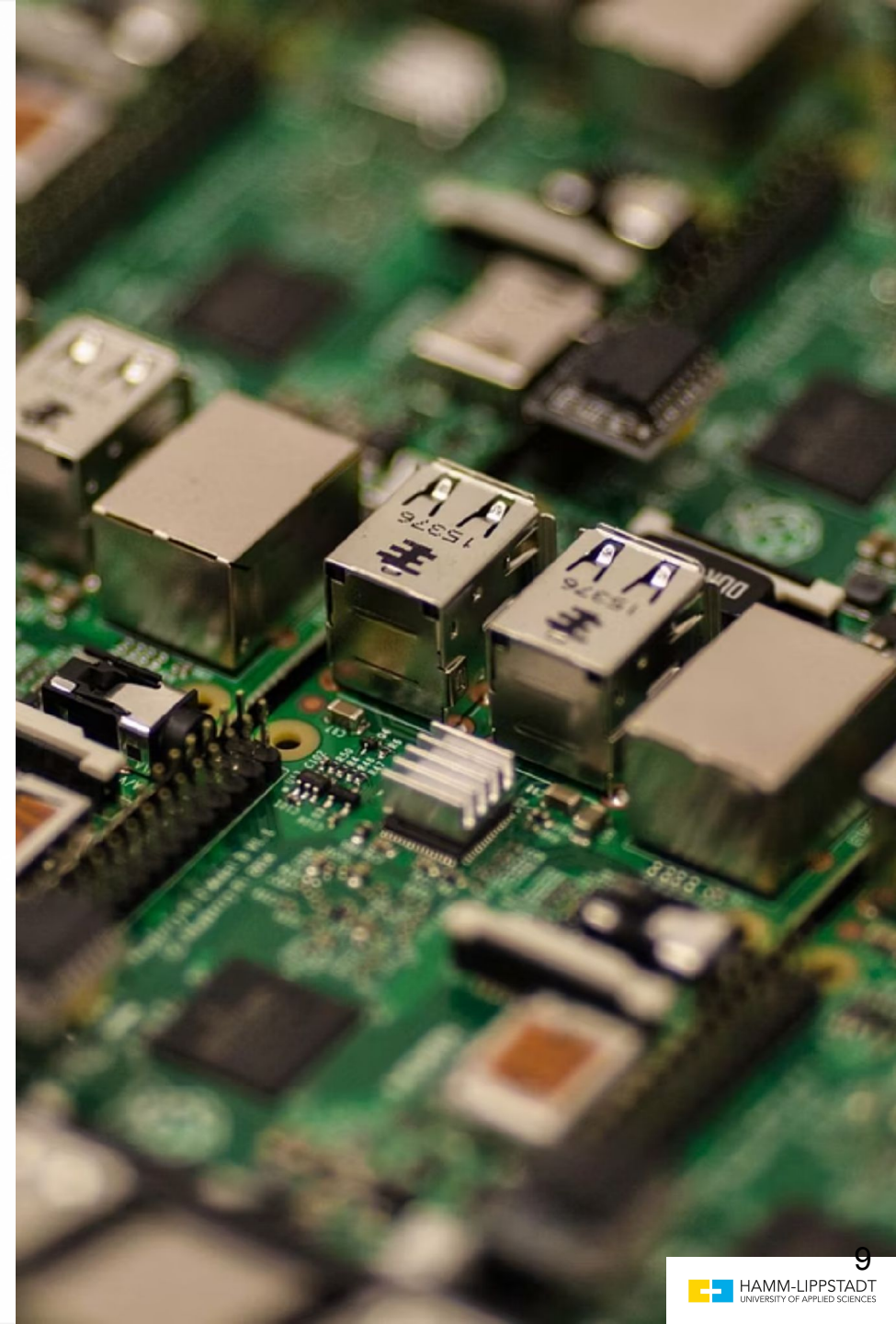
# When to Use PIP – and When Not To

## ✓ Suitable For

- Single-core RTOS
- POSIX legacy migration (PTHREAD\_PRIO\_INHERIT)
- Automotive ECUs with total semaphore ordering
- Mars Pathfinder fix: one config line

## ✗ Not Suitable For

- Multiprocessors — IPI latency breaks timing (Gracioli 2013)
- EDF scheduling — use SRP instead
- Certified deadlock freedom — use PCP
- WCET sensitivity —  $B_i$  only as good as  $\Delta_{j,k}$  inputs (Abella 2015)





# Summary

## Key Takeaways

1. PIP bounds inversion to  $\alpha_i = \min(l_i, s_i)$  CS (Theorem 7.2)
2. Transparency enables deployment without code changes
3. Limitations: no deadlock prevention, chained blocking, hard kernel impl., uniprocessor only

Thank you for your attention, i am open to questions